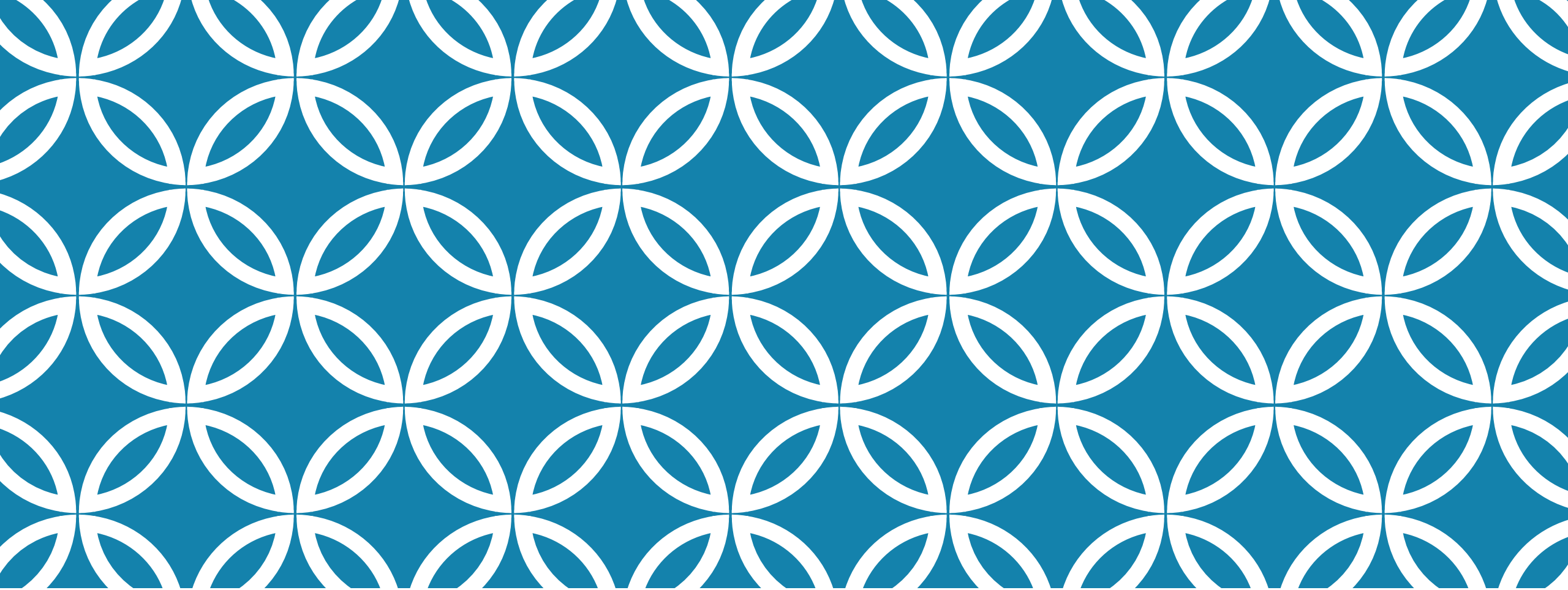


LL(1) PARSER

LL(1) PARSER

Top down parser, predictive parser

first ‘L’ → input is read from left to right

second ‘L’ → produces a left-to-right derivation

The ‘1’ → lookahead of 1 character

The LL(1) parser uses a **stack and a parsing table** to determine the production that should be used for a non-terminal

It **searches a parsing table created** from a specific grammar for the production that should be applied.

It tries to **predict the correct production** to extend the non-terminal at the present derivation step if more than one production relates to the same non-terminal

As a result, to build a predictive parser, we need to **know which production options to use, given the current input symbol and the non-terminal** that has to be extended

Steps:

(1) The grammar must be unambiguous, deterministic and no left recursion

(2) Find the FIRST set

(3) Find the FOLLOW set

(4) Create the LL(1) parser table

(5) Parse the string using the parser table

EXAMPLES

S → aABb

A → c | ε

B → d | ε

Parse the string **acdb$**

Non terminals	FIRST	FOLLOW
S	{a}	{$}
A	{c, ε}	{d,b}
B	{d, ε}	{b}

	a	**b**	**c**	**d**
S	S → aABb			
A		A → ε	A → c	A → ε
B		B → ε		B → d

The grammar is LL(1) grammar (since there is a unique production for each terminal)

Now, let us parse the string **acdb$** (using the parser table created)

Stack	**Input**	**Action**
$	acdb$	Push S
$S	acdb$	S → aABb
$bBAa	acdb$	Pop a
$bBA	cdb$	A → c
$bBc	cdb$	Pop c
$bB	db$	B → d
$bd	db$	Pop d
$b	b$	Pop b
$	$	Accept

The string **acdb$** is valid using the grammar

HOMEWORK : check the string **adb$** is valid or not for this grammar

EXAMPLES

S → AaAb | BbBa

A → ε

B → ε

Non terminals	FIRST	FOLLOW
S	{a, b}	{$}
A	{ε}	{a, b}
B	{ε}	{b, a}

	a	b	c	d
S	S → AaAb	S → BbBa		
A	A → ε	A → ε		
B	B → ε	B → ε		

The grammar is LL(1) grammar (since there is a unique production for each terminal)